Click-to-Source 3D

Stage 1 — Experimental Validation Report

Full Raw Report — All Seven Experiments, Observations, Runtime IDs & Conclusions

Prepared by Punith P — Computer Science & Engineering, KS Institute of Technology, Bengaluru

---

Experiment 1 — Single Cube Provenance Validation

Objective

Determine whether a single runtime Three.js / React Three Fiber mesh can reliably carry manually attached provenance metadata (`sourceRef`) and expose it through click interaction without modification or loss during repeated interaction.

This experiment serves as the foundation for the Click-to-Source 3D architecture. Before testing multiple objects, procedural generation, React lifecycle events, or memoization, it first validates that a runtime mesh can act as a carrier for source provenance.

Research Question

> **Can a runtime mesh reliably carry provenance metadata and return it when selected through user interaction?**

Experimental Setup

Environment

- React
- Vite
- React Three Fiber
- Three.js

Provenance Attachment

The cube was manually tagged with:

```
mesh.userData.sourceRef = {
    file,
    function,
    line,
    args
```

```
}
```

Click Logging

Each click logged:

```
{
    uuid,
    sourceRef,
    timestamp
}
```

No AST transforms, browser extensions, editor integrations, or runtime instrumentation were used.

Expected Result

The following behavior was expected:

- Clicking the cube should resolve the runtime mesh.
- The mesh should expose its attached `sourceRef`.
- The runtime UUID should remain stable across repeated clicks.
- Provenance metadata should remain unchanged.
- Only the timestamp should change between clicks.

No runtime recreation was expected because no React state changes or lifecycle events occurred.

Procedure

1. Start the Vite development server.
2. Render a single cube.
3. Attach provenance metadata.
4. Click the cube five consecutive times.
5. Record every console log.

Raw Console Output

Click 1

```
[Exp1] Click →

uuid:
5709656f-2d9d-4fa9-88e8-e2ee483fe49e

sourceRef:
{ ... }
```

```
timestamp:
1786016383837
```

Click 2

```
[Exp1] Click →

uuid:
5709656f-2d9d-4fa9-88e8-e2ee483fe49e

sourceRef:
{ ... }

timestamp:
1786016387808
```

Click 3

```
[Exp1] Click →

uuid:
5709656f-2d9d-4fa9-88e8-e2ee483fe49e

sourceRef:
{ ... }

timestamp:
1786016388586
```

Click 4

```
[Exp1] Click →

uuid:
5709656f-2d9d-4fa9-88e8-e2ee483fe49e

sourceRef:
{ ... }

timestamp:
1786016389898
```

Click 5

```
[Exp1] Click →

uuid:
5709656f-2d9d-4fa9-88e8-e2ee483fe49e

sourceRef:
{ ... }
```

```
timestamp:
1786016390505
```

Observed Runtime Data

Runtime UUID

```
5709656f-2d9d-4fa9-88e8-e2ee483fe49e
```

Observed on:

- Click 1
- Click 2
- Click 3
- Click 4
- Click 5

The UUID never changed.

Provenance

Each click returned a populated `sourceRef`.

The console confirmed the runtime object carried provenance successfully throughout the experiment.

Although the expanded contents of `sourceRef` were not captured in the console export, it was present and readable on every click.

Timestamp Sequence

Click	Timestamp
1	1786016383837
2	1786016387808
3	1786016388586
4	1786016389898
5	1786016390505

Every click produced a unique timestamp, confirming that each interaction was independently processed while targeting the same runtime object.

Detailed Observations

Object Identity

Across five independent interactions, the runtime UUID remained identical:

```
5709656f-2d9d-4fa9-88e8-e2ee483fe49e
```

No evidence of runtime mesh recreation was observed.

Provenance Persistence

The manually attached `sourceRef` remained attached to the runtime mesh throughout every interaction.

No missing provenance was observed.

No corrupted provenance was observed.

No unexpected mutations occurred.

Raycast Resolution

Each click successfully resolved the intended runtime object.

The raycaster consistently returned the same mesh instance.

Runtime Stability

Since no React state changes, re-renders, remounts, or Hot Module Replacement occurred during this experiment, the runtime environment remained stable.

The experiment therefore validated the baseline behavior under ideal conditions.

Actual Result

Property	Observation
Runtime UUID	Stable
Provenance (`sourceRef`)	Present on every click
Timestamp	Updated every click
Runtime recreation	Not observed
Metadata corruption	Not observed
Raycast resolution	Successful

Result

PASS

Conclusion

Experiment 1 successfully demonstrated that a runtime React Three Fiber / Three.js mesh can carry manually attached provenance metadata through `userData` and expose that metadata during user interaction.

Repeated raycast selection consistently resolved the same runtime object, as evidenced by an unchanged UUID across all five clicks. The attached provenance remained available throughout the experiment, with no observed loss, corruption, or unintended mutation.

This experiment established the fundamental assumption required for the Click-to-Source 3D architecture:

> **A runtime object can act as a reliable carrier of source provenance information.**

Because this assumption held under repeated interaction, the project could proceed to more complex experiments involving multiple generated objects, React lifecycle events, procedural generation, and memoized generation while using the same provenance mechanism.

Experiment 2 — Multiple Generated Cubes (Per-Instance Provenance Validation)

Objective

Determine whether multiple runtime objects generated from the same generator function maintain **independent provenance information** and correctly expose the runtime arguments that created each individual instance.

Unlike Experiment 1, which validated provenance on a single mesh, this experiment investigates whether the same provenance mechanism scales to multiple generated objects without metadata leakage or overwriting between instances.

Research Question

> **Can multiple runtime meshes created by the same generator function each carry their own unique provenance while sharing the same source location?**

Experimental Setup

Environment

- React
- Vite
- React Three Fiber
- Three.js

Generator Function

```
createCube(seed)
```

Each cube was generated through the same function, but invoked with a different runtime seed.

Seeds Used

```
0
1
2
3
4
```

Each generated cube was assigned a unique runtime seed while sharing the same generator function.

Provenance Attachment

Each generated mesh carried:

```
mesh.userData.sourceRef = {
    file,
    function,
    line,
    args
}
```

where

```
args.seed
```

matched the seed used to generate that particular cube.

Click Logging

Each click logged

```
{
    uuid,
    sourceRef,
    timestamp
}
```

Expected Result

The following behavior was expected:

- Every cube should have its own runtime UUID.
- Every cube should expose a valid `sourceRef`.
- Every cube should report:
- identical source file
- identical generator function

- identical source line
- Only the runtime arguments (`seed`) should differ.
- No provenance should leak between instances.

Procedure

1. Generate five cubes using the same generator function.
2. Assign seeds:

```
0
1
2
3
4
```

3. Render all five cubes.
4. Click each cube exactly once.
5. Record every console log.

Raw Console Output

Cube 1

```
[Exp2] Click →

uuid:
b08443f8-2167-4313-8b17-7d738d1f37d0

sourceRef

file:
Exp2_MultipleCubes.jsx

function:
createCube

line:
12

args:
{
    seed: 0
}

timestamp:
1786016475950
```

Cube 2

```
[Exp2] Click →

uuid:
8a143450-a7a8-4249-8f98-71319ed6c33c

sourceRef

file:
Exp2_MultipleCubes.jsx

function:
createCube

line:
12

args:
{
    seed: 1
}

timestamp:
1786016477755
```

Cube 3

```
[Exp2] Click →

uuid:
a29eaded-fde2-478e-bd46-f06a3ccf805b

sourceRef

file:
Exp2_MultipleCubes.jsx

function:
createCube

line:
12

args:
{
    seed: 2
}

timestamp:
1786016479607
```

Cube 4

```
[Exp2] Click →
```

```
uuid:
a8c948fd-4cf9-46f3-a8a8-09ca75d377bb

sourceRef

file:
Exp2_MultipleCubes.jsx

function:
createCube

line:
12

args:
{
    seed: 3
}

timestamp:
1786016480793
```

Cube 5

The fifth console log was truncated during capture.

The available evidence confirms that it reported:

```
seed:
4
```

However, the UUID and timestamp were not preserved in the exported console log and therefore cannot be reported without guessing.

Observed Runtime Data

Cube	UUID	Seed
1	`b08443f8-2167-4313-8b17-7d738d1f37d0`	0
2	`8a143450-a7a8-4249-8f98-71319ed6c33c`	1
3	`a29eaded-fde2-478e-bd46-f06a3ccf805b`	2
4	`a8c948fd-4cf9-46f3-a8a8-09ca75d377bb`	3
5	*(UUID not captured)*	4

Static Provenance

Every successfully captured click returned the same static provenance:

```
file:
Exp2_MultipleCubes.jsx
```

```
function:
createCube

line:
12
```

Only the runtime argument

```
seed
```

changed between objects.

Runtime Arguments

Cube	Runtime Arguments
Cube 1	`{ seed: 0 }`
Cube 2	`{ seed: 1 }`
Cube 3	`{ seed: 2 }`
Cube 4	`{ seed: 3 }`
Cube 5	`{ seed: 4 }`

Detailed Observations

Generator Provenance

All cubes originated from the same generator function.

Every runtime object reported

```
function:
createCube
```

demonstrating that provenance consistently resolved to the generator responsible for object creation.

Source Location

Every captured object pointed to the same source location:

```
Exp2_MultipleCubes.jsx

line 12
```

This indicates that provenance identifies the generator definition rather than treating each runtime object as a different source location.

Runtime Differentiation

Although all cubes shared identical static provenance, each object carried different runtime arguments through

```
args.seed
```

The recovered seeds matched the values used during generation:

```
0
1
2
3
4
```

This confirms that provenance remained instance-specific.

Object Identity

Each captured runtime object possessed a unique UUID.

No duplicated UUIDs were observed among the recorded objects.

This confirms that React Three Fiber created distinct runtime mesh instances for each generated cube.

Provenance Isolation

No cube reported another cube's seed.

No evidence of metadata leakage, overwriting, or cross-instance contamination was observed.

Each runtime object preserved only the provenance associated with its own generator invocation.

Actual Result

Property	Observation
Generator function	Shared across all cubes
Source file	Shared
Source line	Shared
Runtime UUID	Unique per captured cube
Runtime arguments	Unique per cube
Metadata leakage	Not observed
Provenance corruption	Not observed

Result

PASS

Conclusion

Experiment 2 successfully demonstrated that the provenance mechanism scales beyond a single runtime object.

Although all five cubes originated from the same generator function and shared the same source location, each runtime mesh preserved its own independent provenance through its runtime arguments. The recovered `seed` values matched the generator inputs for each individual cube, confirming that provenance remained instance-specific rather than being shared or overwritten.

The experiment also verified that each generated cube existed as an independent runtime object with its own UUID, while correctly resolving back to the common generator definition.

This experiment established one of the core architectural requirements for Click-to-Source 3D:

> **Multiple objects generated by the same source code can coexist while retaining unique runtime provenance, allowing individual instances to be traced back to both their shared generator and the specific runtime arguments that produced them.**

Note

The console output for the fifth cube was truncated during capture. The available evidence confirms that it reported `seed: 4`; however, its UUID and timestamp were not preserved. These values have been intentionally omitted rather than reconstructed to maintain the integrity of the experimental record.

Experiment 3 — React Parent Re-render (Provenance Stability)

Objective

Determine whether an **ordinary React parent re-render** affects the runtime identity of a React Three Fiber mesh or causes any loss of provenance information.

Unlike Experiment 2, which focused on multiple generated objects, this experiment investigates whether unrelated React state updates trigger recreation of runtime meshes or disturb the attached provenance metadata.

Research Question

> **Does an ordinary React parent re-render recreate the runtime mesh, or does the existing runtime object retain its identity and provenance?**

Experimental Setup

Environment

- React
- Vite
- React Three Fiber
- Three.js

Generator Function

```
createCube(seed)
```

A single cube was generated using:

```
seed = 0
```

Provenance Attachment

The runtime mesh carried:

```
mesh.userData.sourceRef = {
    file,
    function,
    line,
    args
}
```

Re-render Trigger

An **unrelated React state variable** (counter button) was introduced.

The counter was **not connected** to cube generation or provenance.

Its only purpose was to trigger a parent React re-render.

Click Logging

Each click logged:

```
{
    uuid,
    sourceRef,
    timestamp
}
```

Expected Result

The following behavior was expected:

- React should re-render the parent component.
- The runtime `THREE.Mesh` should **not** be recreated.
- The runtime UUID should remain unchanged.

- Provenance should remain attached.
- Only the timestamp should change.

Since the cube generation inputs remain unchanged, React reconciliation is expected to preserve the existing runtime mesh.

Procedure

1. Render one cube.
2. Click the cube and record its provenance.
3. Increment the unrelated counter.
4. React performs a parent re-render.
5. Click the same cube again.
6. Compare UUID and provenance before and after the re-render.

Raw Console Output

Before Parent Re-render

```
[Exp3] Click →

uuid:
90bd3023-07ba-4c41-a207-c5e65e31e2a0

sourceRef

file:
Exp3_ReRender.jsx

function:
createCube

line:
11

args:
{
    seed: 0
}

timestamp:
1786016741833
```

After Parent Re-render

```
[Exp3] Click →
```

```
uuid:
90bd3023-07ba-4c41-a207-c5e65e31e2a0

sourceRef

file:
Exp3_ReRender.jsx

function:
createCube

line:
11

args:
{
    seed: 0
}

timestamp:
1786016745637
```

Observed Runtime Data

Property	Before Re-render	After Re-render
UUID	`90bd3023-07ba-4c41-a207-c5e65e31e2a0`	`90bd3023-07ba-4c41-a207-c5e65e31e2a0`
File	`Exp3_ReRender.jsx`	`Exp3_ReRender.jsx`
Function	`createCube`	`createCube`
Line	`11`	`11`
Seed	`0`	`0`
Timestamp	`1786016741833`	`1786016745637`

Detailed Observations

Runtime Object Identity

The runtime UUID remained identical before and after the parent re-render.

```
90bd3023-07ba-4c41-a207-c5e65e31e2a0
```

No runtime mesh recreation was observed.

This indicates that React reconciliation preserved the existing `THREE.Mesh` instance rather than destroying and rebuilding it.

Provenance Stability

The attached provenance remained completely unchanged.

The runtime mesh continued to report:

```
file:
Exp3_ReRender.jsx

function:
createCube

line:
11

args:
{
    seed: 0
}
```

No provenance loss occurred.

No metadata mutation occurred.

No unexpected values appeared after the re-render.

React Lifecycle Behavior

Incrementing the unrelated counter successfully triggered a React parent re-render.

However, because the cube's props and generation inputs did not change, React Three Fiber reused the existing runtime mesh.

This demonstrates that ordinary React rendering does not inherently recreate Three.js objects.

Timestamp Behavior

The timestamp changed between clicks:

Before	After
1786016741833	1786016745637

This confirms that two separate user interactions occurred while targeting the same runtime object.

Actual Result

Property	Observation
Parent re-render	Occurred successfully
Runtime UUID	Unchanged
Runtime mesh recreation	Not observed
Provenance	Preserved

Metadata corruption	Not observed
Runtime arguments	Unchanged
Raycast resolution	Successful before and after

Result

PASS

Conclusion

Experiment 3 successfully demonstrated that an **ordinary React parent re-render does not recreate the underlying React Three Fiber runtime mesh** when the mesh's generation inputs remain unchanged.

The runtime UUID remained constant before and after the re-render, indicating that the same `THREE.Mesh` instance continued to exist throughout the React lifecycle event. The attached provenance metadata also remained intact, with identical source location and runtime arguments reported on both interactions.

This experiment validates an important architectural assumption for Click-to-Source 3D:

> **Ordinary React rendering does not disturb runtime provenance. Existing runtime objects can safely retain their attached provenance metadata across parent component re-renders, provided the underlying object is not explicitly recreated.**

This establishes React reconciliation as a **non-destructive lifecycle boundary** for the provenance mechanism, allowing subsequent experiments to focus on scenarios where runtime object recreation actually occurs, such as key-based remounting and Hot Module Replacement.

Experiment 4 — React Mesh Recreation (Key-Based Remount)

Objective

Determine how provenance behaves when React is explicitly instructed to destroy and recreate a runtime mesh using the `key` prop.

Unlike Experiment 3, which investigated an ordinary parent re-render, this experiment intentionally forces React to discard the existing component instance and construct a new one. The goal is to determine whether provenance is preserved across runtime object recreation and whether a newly created mesh receives fresh provenance information.

Research Question

When React recreates a runtime mesh through a key change, does the newly created mesh receive correct provenance information, or is provenance lost during object recreation?

Experimental Setup

Environment

- React
- Vite
- React Three Fiber
- Three.js

Component

A single cube component was rendered using:

```
<mesh key={version}>
```

The `version` state variable controlled the React `key`.

Incrementing `version` forced React to:

- Destroy the existing component.
- Create a completely new runtime component.
- Construct a new runtime mesh.

Provenance Attachment

The recreated mesh received provenance through:

```
mesh.userData.sourceRef = {
    file,
    function,
    line,
    args
}
```

where

```
args.version
```

matched the current version value.

Click Logging

Each click logged:

```
{
    uuid,
```

```
    sourceRef,
    timestamp
}
```

Expected Result

The following behavior was expected:

- Changing the React `key` should force a component remount.
- The original runtime mesh should be destroyed.
- A new runtime mesh should receive a new UUID.
- Provenance should be reapplied to the new runtime mesh.
- The new provenance should contain the updated runtime argument (`version`).

Unlike Experiment 3, runtime recreation **was expected**.

Procedure

1. Render a cube with:

```
version = 1
```

2. Click the cube.

3. Record:

- UUID
- provenance
- timestamp

4. Increment:

```
version
```

5. React recreates the component.

6. Click the new cube.

7. Compare UUIDs and provenance.

Raw Console Output

Before Recreation

```
[Exp4] Click →

uuid:
c743f578-747a-4387-a6f3-e82b64d21d6a
```

```
sourceRef

file:
Exp4_MeshRecreation.jsx

function:
RecreatableCube

line:
12

args:
{
    version: 1
}

timestamp:
1786017055004
```

After Recreation

```
[Exp4] Click →

uuid:
c007e247-ad23-484a-bbd0-bae5b16dcd74

sourceRef

file:
Exp4_MeshRecreation.jsx

function:
RecreatableCube

line:
12

args:
{
    version: 2
}

timestamp:
1786017061008
```

Observed Runtime Data

Property	Before Recreation	After Recreation
UUID	`c743f578-747a-4387-a6f3-e82b64d21d6a`	`c007e247-ad23-484a-bbd0-bae5b16dcd74`
File	`Exp4_MeshRecreation.jsx`	`Exp4_MeshRecreation.jsx`
Function	`RecreatableCube`	`RecreatableCube`
Line	`12`	`12`

Version	1	2
Timestamp	1786017055004	1786017061008

Detailed Observations

Runtime Object Identity

The runtime UUID changed completely after incrementing the React `key`.

Before recreation:

```
c743f578-747a-4387-a6f3-e82b64d21d6a
```

After recreation:

```
c007e247-ad23-484a-bbd0-bae5b16dcd74
```

This confirms that React destroyed the original runtime mesh and created an entirely new `THREE.Mesh`.

Unlike Experiment 3, object identity was **not preserved**.

Provenance Reconstruction

Despite the complete runtime recreation, the new mesh immediately exposed valid provenance.

The recreated mesh reported:

```
file:
Exp4_MeshRecreation.jsx

function:
RecreatableCube

line:
12
```

indicating that provenance was successfully attached during creation of the new runtime object.

Runtime Argument Update

The provenance also reflected the updated runtime state.

Before recreation:

```
version = 1
```

After recreation:

```
version = 2
```

This confirms that provenance was not copied from the previous runtime object but instead reconstructed using the current runtime arguments.

React Lifecycle Behavior

Changing the `key` forced React's reconciliation algorithm to treat the component as a completely new instance.

The previous runtime mesh was discarded.

A fresh runtime mesh was created.

The provenance mechanism executed again during creation, resulting in a correctly tagged replacement object.

Timestamp Behavior

The timestamps differed:

Before	After
1786017055004	1786017061008

confirming two independent runtime interactions.

Actual Result

Property	Observation
React remount	Successfully triggered
Runtime UUID	Changed
Runtime mesh recreation	Observed
Provenance	Present before and after
Source location	Unchanged
Runtime arguments	Updated correctly
Metadata corruption	Not observed

Result

PASS

Conclusion

Experiment 4 successfully demonstrated that **React key-based remounting destroys the original runtime mesh and creates a new runtime object**, as evidenced by the change in UUID between the two interactions.

Although the original mesh ceased to exist, the newly created runtime mesh immediately received valid provenance information during its creation. The recovered source location remained identical, while the runtime argument (`version`) correctly reflected the new component state.

This experiment establishes an important architectural property of Click-to-Source 3D:

> **The provenance mechanism does not depend on preserving runtime object identity. Instead, provenance is reattached whenever React creates a new runtime object.**

Unlike Experiment 3, where React preserved the existing mesh during reconciliation, Experiment 4 confirms that the provenance system remains reliable even when runtime objects are explicitly destroyed and recreated, provided provenance is attached as part of the object's creation lifecycle.

Experiment 5 — Hot Module Replacement (HMR) Lifecycle Validation

---

Objective

Determine how provenance behaves when the application undergoes **Vite Hot Module Replacement (HMR)** during development.

Unlike Experiment 4, where React was explicitly instructed to recreate the runtime mesh through a `key` change, this experiment investigates whether a development-time hot update preserves or recreates runtime objects and whether provenance remains available afterward.

The objective is to understand how Click-to-Source 3D behaves during active development, where source files are frequently modified without performing a full browser refresh.

Research Question

> **Does a Vite Hot Module Replacement preserve runtime provenance, or does recreating runtime objects during HMR cause provenance to be lost?**

Experimental Setup

Environment

- React
- Vite
- React Three Fiber
- Three.js

Component

A single tagged cube was rendered.

The cube carried manually attached provenance through:

```
mesh.userData.sourceRef = {
    file,
    function,
    line,
    args
}
```

Click Logging

Each click logged:

```
{
    uuid,
    sourceRef,
    timestamp
}
```

HMR Trigger

A harmless, unrelated edit was made to the source file in order to trigger Vite Hot Module Replacement.

The experiment intended to observe runtime behavior **without performing a full browser refresh**.

Expected Result

The following behavior was expected:

- Vite should perform a Hot Module Replacement.
- React may either preserve or recreate the runtime mesh.
- Provenance should remain available regardless of runtime object recreation.
- Source location should remain identical before and after HMR.
- Runtime arguments should remain unchanged.

The experiment was intended to determine whether HMR introduces any architectural risk to the provenance mechanism.

Procedure

1. Start the Vite development server.

2. Render a tagged cube.

3. Click the cube.

4. Record:

- UUID
- provenance
- timestamp

5. Modify an unrelated portion of the source file.

6. Trigger Vite Hot Module Replacement.

7. Click the cube again.

8. Compare runtime identity and provenance.

Raw Console Output (Original Experiment)

Before HMR

```
[Exp5] Click →

uuid:
a56b0a43-79e5-42f1-8b5a-70002961725b

sourceRef

file:
Exp5_HMR.jsx

function:
TaggedCube

line:
26

args:
{
    color: "dodgerblue",
    purpose: "hmr-test"
}

timestamp:
1786017344077
```

After Original Test

```
[Exp5] Click →

uuid:
3877031b-9392-4ac1-b818-b4c131c9034b

sourceRef

file:
Exp5_HMR.jsx
```

```
function:
TaggedCube

line:
26

args:
{
    color: "dodgerblue",
    purpose: "hmr-test"
}

timestamp:
1786017433904
```

Observed Runtime Data (Original Execution)

Property	Before	After
UUID	`a56b0a43-79e5-42f1-8b5a-70002961725b`	`3877031b-9392-4ac1-b818-b4c131c9034b`
File	`Exp5_HMR.jsx`	`Exp5_HMR.jsx`
Function	`TaggedCube`	`TaggedCube`
Line	`26`	`26`
Runtime Arguments	Identical	Identical

Initial Observation

The runtime UUID changed.

This suggested that the runtime mesh had been recreated.

The attached provenance remained available after recreation.

At the time of the experiment, this appeared to indicate that the provenance mechanism continued to function after HMR.

Correction Note (Post-Stage 1 Review)

Following completion of Stage 1, the methodology for Experiment 5 was reviewed.

It was determined that the original execution mistakenly involved a **browser hard refresh** while attempting to validate Hot Module Replacement.

A browser hard refresh destroys the entire JavaScript runtime, reloads the page, and recreates every runtime object. Therefore, it is **not equivalent to Vite Fast Refresh / Hot Module Replacement**.

As a result, the UUID transition observed during the original execution:

```
a56b0a43-79e5-42f1-8b5a-70002961725b

↓

3877031b-9392-4ac1-b818-b4c131c9034b
```

cannot be interpreted as evidence of genuine HMR behavior.

Authoritative HMR Validation

A genuine Vite Hot Module Replacement was later performed during **Experiment 7**.

Experiment 7 observed:

- Vite emitted:

```
[vite] hot updated:
/src/Exp7_MemoizedGeneration.jsx
```

- The component re-rendered.
- `useMemo` executed again.
- Provenance remained available after the hot update.
- Runtime interaction continued successfully without loss of provenance.

Experiment 7 therefore provides the authoritative validation of Hot Module Replacement behavior for Stage 1.

Actual Result

Property	Observation
Runtime recreation observed	Yes
Provenance remained attached	Yes
Source location	Preserved
Runtime arguments	Preserved
Methodology	Later determined to include a hard refresh rather than pure HMR

Result

PASS (Superseded by Experiment 7)

The experiment successfully demonstrated that provenance was preserved across runtime recreation.

However, because the runtime recreation occurred after a browser hard refresh rather than a genuine Hot Module Replacement, the experiment **does not constitute authoritative evidence for HMR behavior**.

The definitive HMR validation is provided by Experiment 7.

Conclusion

Experiment 5 initially suggested that provenance survived runtime recreation following what was believed to be a Hot Module Replacement event.

Subsequent review of the experimental methodology revealed that a browser hard refresh had been performed during testing. Because a hard refresh reconstructs the entire application from scratch, it cannot be used to characterize the lifecycle behavior of Vite Hot Module Replacement.

Accordingly, Experiment 5 remains included in the Stage 1 record as part of the chronological development process, but its conclusions regarding HMR have been superseded by the controlled HMR validation performed in Experiment 7.

The primary architectural takeaway remains valid:

> **Runtime object recreation did not result in the loss of provenance information.**

However, the specific claim regarding Hot Module Replacement is based on **Experiment 7**, not on the original execution of Experiment 5.

Methodological Note

This correction has been intentionally preserved in the experimental record rather than removing or rewriting the original experiment. Maintaining the chronological record of the investigation, including methodological corrections, provides a transparent account of how the architecture was validated and reflects standard engineering research practice.

Experiment 6 — Procedural Tree Generation (Realistic Generator Validation)

Objective

Determine whether the provenance mechanism continues to function correctly when applied to a **more realistic procedural generation pattern** rather than simple cubes.

Unlike Experiments 1–5, which used intentionally simple cube primitives, this experiment introduces a generator function resembling the structure of an actual procedural content pipeline. The objective is to verify that provenance remains attached to runtime objects created by a generator accepting multiple runtime parameters.

This experiment acts as the first approximation of how the Click-to-Source 3D architecture will operate within a procedural world generator.

Research Question

Can a procedural generator function create multiple runtime objects that each preserve independent provenance while sharing the same generator definition?

Experimental Setup

Environment

- React
- Vite
- React Three Fiber
- Three.js

Generator Function

```
createTree(x, z, seed)
```

Unlike previous experiments, the generator accepted **three runtime parameters**:

- `x`
- `z`
- `seed`

Each invocation simulated a procedural tree being generated at a unique world position.

Procedural Generation

Five trees were generated.

Each invocation used different runtime arguments.

Although every tree originated from the same generator function, each runtime object carried its own independent provenance.

Provenance Attachment

Each generated tree received:

```
mesh.userData.sourceRef = {
    file,
    function,
    line,
    args
}
```

where

```
args = {
    x,
```

```
    z,
    seed
}
```

Click Logging

Each click logged:

```
{
    uuid,
    sourceRef,
    timestamp
}
```

Expected Result

The following behavior was expected:

- Every generated tree should possess its own runtime UUID.
- Every tree should expose valid provenance.
- Every tree should report:
- identical source file
- identical generator function
- identical source line
- Runtime arguments should differ for every tree.
- Provenance should remain isolated between instances.

Unlike Experiment 2, this experiment validates provenance using a generator signature representative of real procedural content generation.

Procedure

1. Generate five procedural trees.

2. Each tree is created using:

```
createTree(x, z, seed)
```

3. Render all trees.

4. Click each tree once.

5. Record:

- UUID
- provenance

- runtime arguments
- timestamp

6. Compare provenance across all generated objects.

Raw Console Output

Tree 1

```
[Exp6] Click →

uuid:
91cc4ba5-2c3c-4d41-bd93-28d5939c6b64

sourceRef

file:
Exp6_Trees.jsx

function:
createTree

line:
18

args:
{
    x: -4,
    z: 0,
    seed: 100
}

timestamp:
1786019419500
```

Tree 2

```
[Exp6] Click →

uuid:
72f6b9fc-0d58-4eb7-bf84-9db47c5f643d

sourceRef

file:
Exp6_Trees.jsx

function:
createTree

line:
18
```

```
args:
{
    x: -2,
    z: 0,
    seed: 101
}

timestamp:
1786019421646
```

Tree 3

```
[Exp6] Click →

uuid:
4d6bf497-7196-44ea-a991-f7d367b5b95b

sourceRef

file:
Exp6_Trees.jsx

function:
createTree

line:
18

args:
{
    x: 0,
    z: 0,
    seed: 102
}

timestamp:
1786019423297
```

Tree 4

```
[Exp6] Click →

uuid:
64a16ca5-b53f-4d9b-ae97-c89af6db0fc4

sourceRef

file:
Exp6_Trees.jsx

function:
createTree

line:
18
```

```
args:
{
    x: 2,
    z: 0,
    seed: 103
}

timestamp:
1786019424937
```

Tree 5

```
[Exp6] Click →

uuid:
1b4af57b-8c77-4bd6-83fa-820f97546ec6

sourceRef

file:
Exp6_Trees.jsx

function:
createTree

line:
18

args:
{
    x: 4,
    z: 0,
    seed: 104
}

timestamp:
1786019426456
```

Observed Runtime Data

Tree	UUID	x	z	Seed
1	91cc4ba5-2c3c-4d41-bd93-28d5939c6b64	-4	0	100
2	72f6b9fc-0d58-4eb7-bf84-9db47c5f643d	-2	0	101
3	4d6bf497-7196-44ea-a991-f7d367b5b95b	0	0	102
4	64a16ca5-b53f-4d9b-ae97-c89af6db0fc4	2	0	103
5	1b4af57b-8c77-4bd6-83fa-820f97546ec6	4	0	104

Static Provenance

Every tree reported identical static provenance:

```
file:
Exp6_Trees.jsx

function:
createTree

line:
18
```

This confirms that all runtime objects originated from the same procedural generator.

Runtime Arguments

Each tree carried unique runtime generation arguments.

Tree	Runtime Arguments
Tree 1	`{ x: -4, z: 0, seed: 100 }`
Tree 2	`{ x: -2, z: 0, seed: 101 }`
Tree 3	`{ x: 0, z: 0, seed: 102 }`
Tree 4	`{ x: 2, z: 0, seed: 103 }`
Tree 5	`{ x: 4, z: 0, seed: 104 }`

Detailed Observations

Procedural Provenance

Every generated tree successfully reported provenance pointing to the procedural generator responsible for its creation.

All trees identified:

```
function:
createTree
```

demonstrating that provenance remained attached regardless of the number of generated objects.

Runtime Differentiation

Although all trees shared the same generator definition, each runtime object preserved its own procedural parameters.

The recovered values for:

- `x`
- `z`

- `seed`

exactly matched the arguments supplied during generation.

This demonstrates that provenance remained **instance-specific**, even within a procedural generation pipeline.

Runtime Object Identity

Every tree possessed its own runtime UUID.

No duplicate UUIDs were observed.

Each generated tree therefore existed as an independent runtime object.

Provenance Isolation

No tree reported another tree's runtime parameters.

No metadata leakage occurred.

No runtime arguments were overwritten.

Each generated object preserved only the provenance associated with its own generator invocation.

Architectural Significance

This experiment represents the first validation using a generator signature similar to that used in procedural world generation.

Unlike the previous cube experiments, provenance now carried multiple runtime parameters that together describe the generated object.

This more closely resembles the intended architecture of Click-to-Source 3D, where runtime objects are expected to preserve the generation context responsible for their creation.

Actual Result

Property	Observation
Generator function	Shared across all trees
Source file	Shared
Source line	Shared
Runtime UUID	Unique per tree
Runtime arguments	Unique per tree
Metadata leakage	Not observed
Provenance corruption	Not observed

Procedural generation	Successfully validated

Result

PASS

Conclusion

Experiment 6 successfully demonstrated that the provenance mechanism extends beyond simple primitive objects and remains reliable within a procedural generation workflow.

Although every tree originated from the same procedural generator (`createTree`), each runtime object preserved its own independent runtime context (`x`, `z`, and `seed`) while continuing to resolve back to the common source definition.

This validates one of the central architectural assumptions of Click-to-Source 3D:

> **Procedurally generated runtime objects can preserve both their shared source origin and their unique generation parameters, allowing individual instances to be traced back to the exact generator invocation that created them.**

Experiment 6 therefore provides the first realistic validation that the provenance mechanism is applicable to procedural world generation rather than only simple demonstration primitives.

The next experiment is **Experiment 7**, which is the final experiment of Stage 1.

Experiment 7 — Memoized Generation Lifecycle Validation

Objective

Determine whether the provenance mechanism remains reliable when generation is **memoized using React's** `useMemo`, rather than being recreated on every render.

Unlike Experiments 1–6, where provenance was attached through JSX on each render, this experiment models a more realistic procedural generation pipeline. Expensive generation is cached using `useMemo`, with provenance attached during object creation inside the memoized callback. The experiment investigates whether provenance remains correct across parent re-renders, dependency changes, and Vite Hot Module Replacement.

Research Question

Does memoized procedural generation preserve correct provenance across React lifecycle events, or does it require a different tagging strategy than the declarative JSX approach validated in previous experiments?

Experimental Setup

Environment

- React
- Vite
- React Three Fiber
- Three.js

Generator Function

```
createCube(seed)
```

Unlike previous experiments, `createCube()` **does not create a** `THREE.Mesh`. It generates a plain JavaScript object containing the procedural generation data and provenance metadata.

```
{
    color,
    sourceRef
}
```

The runtime `THREE.Mesh` is created declaratively by React Three Fiber through JSX.

Memoization

Generation data was wrapped in:

```
useMemo(() => {
    return createCube(seed)
}, [seed])
```

Only `seed` appears in the dependency array.

`version` intentionally does **not** invalidate the memo.

Provenance Attachment

Provenance is created **inside the** `useMemo` **callback**:

```
sourceRef = {
    file,
    function,
    line,
    args
}
```

and attached to the mesh through

```
<mesh userData={{ sourceRef: cubeData.sourceRef }}>
```

Object Identity Instrumentation

The experiment compared the identity of the **memoized generation object (**`cubeData`**)**, **not** the underlying `THREE.Mesh`.

Literal comparison used:

```
const isSameObject = prevDataRef.current === cubeData
```

This comparison measures whether React reused the memoized generation object returned by `useMemo`.

It **does not** compare runtime mesh identity.

Click Logging

Each click logged:

```
{
    uuid: e.object.uuid,
    sourceRef: e.object.userData.sourceRef,
    timestamp: Date.now(),
    objectReused: isSameObject
}
```

The UUID comes directly from the raycast hit:

```
e.object.uuid
```

Expected Result

The following behavior was expected:

Step	Expected Behaviour
Baseline	Provenance available
Version change	useMemo should NOT execute
Seed change	useMemo SHOULD execute
HMR	Behaviour unknown (research objective)

Procedure

Step 1

Click cube immediately after startup.

Step 2

Increment Version.

Click cube again.

Step 3

Increment Seed.

Click cube again.

Step 4

Trigger Vite Hot Module Replacement by editing an unrelated string.

Click cube again.

Raw Experimental Output

Step 1 — Initial Mount

Parent Render

```
seed = 1
version = 1
hmrMarker = exp7-hmr-trigger-v1
```

(two renders)

---

useMemo

```
[Exp7] useMemo executed

seed:
1

timestamp:
1786018925404

[Exp7] useMemo executed

seed:
1

timestamp:
1786018925405
```

Object Identity

```
Render — initial mount
```

followed by

```
reused:
true
```

Click

UUID

```
72de18fd-651a-4e75-87f2-5063ad6976f8
```

Timestamp

```
1786018930782
```

Provenance

```
file:
Exp7_MemoizedGeneration.jsx

function:
createCube

line:
24

seed:
1
```

Step 2 — Increment Version

Parent render occurred.

```
seed = 1

version = 2
```

No `useMemo executed` log appeared.

Object identity

```
reused:
true
```

Click

UUID

```
72de18fd-651a-4e75-87f2-5063ad6976f8
```

Timestamp

```
1786018980368
```

Provenance

```
seed:
1
```

Step 3 — Increment Seed

Parent render

```
seed = 2

version = 2
```

`useMemo` executed again.

```
seed:
2

timestamp:
1786019030453
```

Object identity

```
reused:
false
```

then

```
reused:
true
```

Click

UUID

```
72de18fd-651a-4e75-87f2-5063ad6976f8
```

Timestamp

```
1786019031870
```

Provenance

```
seed:
2
```

Step 4 — Genuine Vite HMR

Vite reported

```
[vite] hot updated:
/src/Exp7_MemoizedGeneration.jsx
```

Parent render

```
seed = 2

version = 2

hmrMarker = exp7-hmr-trigger-v3
```

`useMemo` executed again.

```
seed:
2

timestamp:
1786019200707
```

and

```
seed:
2

timestamp:
1786019200709
```

Object identity

```
reused:
false
```

followed by

```
reused:
true
```

Click

UUID

```
72de18fd-651a-4e75-87f2-5063ad6976f8
```

Timestamp

```
1786019287847
```

Provenance

```
file:
Exp7_MemoizedGeneration.jsx

function:
createCube

line:
24

seed:
2
```

Detailed Observations

Memoization Behaviour

Changing **version** did not invalidate the memo.

`useMemo` was **not executed**.

Changing **seed** invalidated the memo exactly as expected.

`useMemo` executed again and created a new `cubeData` object.

Object Identity

The experiment compared:

```
prevDataRef.current === cubeData
```

This measures reuse of the **memoized generation object**, **not** the runtime mesh.

When `seed` changed, the comparison returned:

```
false
```

confirming a new memoized generation object was created.

Runtime Mesh Identity

Throughout all four stages, the clicked mesh reported the same UUID:

```
72de18fd-651a-4e75-87f2-5063ad6976f8
```

This indicates that React Three Fiber preserved the underlying `THREE.Mesh` while updating its associated generation data.

The stable UUID is **not contradictory** to the `reused` flag because the two values describe different layers of the system.

Provenance

At every stage, the clicked runtime mesh returned valid provenance.

After changing the seed, provenance correctly updated from:

```
seed:
1
```

to

```
seed:
2
```

No provenance loss occurred during:

- parent re-render
- memo invalidation
- genuine Vite HMR

Actual Result

Property	Observation
Parent re-render	Memo reused
Version increment	`useMemo` not executed
Seed increment	`useMemo` executed
HMR	`useMemo` executed
Runtime mesh UUID	Stable throughout
Memoized generation object	Recreated when seed changed
Provenance	Present throughout
Metadata corruption	Not observed

Result

PASS

Conclusion

Experiment 7 successfully validated the provenance mechanism under a memoized procedural generation workflow.

The experiment demonstrated that React's `useMemo` controls the lifecycle of the **generation data (`cubeData`)**, while React Three Fiber independently manages the lifecycle of the underlying `THREE.Mesh`.

Changing the memo dependency (`seed`) recreated the memoized generation object, whereas the runtime mesh retained the same UUID. This shows that memoized generation data and runtime mesh identity are separate concerns.

A genuine Vite Hot Module Replacement caused the memoized generation callback to execute again, after which valid provenance remained attached to the runtime mesh. No provenance loss was observed during parent re-renders, dependency changes, or HMR.

This experiment significantly strengthens the architectural confidence in Click-to-Source 3D by demonstrating that the provenance mechanism remains reliable even when procedural generation is cached using `useMemo`, closely matching the intended architecture of real-world procedural generation systems.